CodeLens AI

Technical Architecture & Core Codebase Documentation

Version: 1.0.0 (Release-Ready)

Prepared For: CodeLens AI Engineering Team

Date: July 2026

1. Executive Summary

CodeLens AI is a modern developer tool built on top of Spring Boot and Java 21. It acts as a smart codebase analysis engine that ingests Java repositories from GitHub, parses class and method-level structures to build an Abstract Syntax Tree (AST), tracks callers and callees to model dependency graphs, and generates vector embeddings for semantic codebase searching.

By combining a relational database (PostgreSQL) for structured metadata/relationships and a vector database (pgvector) for semantic code search, CodeLens AI enables developers to ask natural language questions about their code (e.g., 'Find all payment verification functions') and navigate code structures dynamically.

2. System Architecture & Component Mapping

The application follows a standard modular design. When a client submits a public GitHub repository URL, the indexing process is offloaded to a background executor, allowing the API request to return a 202 Accepted status immediately. The backend uses the following components:

GitHub Ingestion Service (GitHubService)

Fetches files recursively from public GitHub repositories using the GitHub Contents API. It filters for files ending with '.java', retrieves their base64-encoded file blobs, decodes them, and translates them into flat in-memory source string records.

Java AST Service (JavaAstService)

Uses the JavaParser library to build an Abstract Syntax Tree (AST) for each file. It identifies all classes (skipping interfaces) and concrete method declarations. Crucially, it traverses method bodies to identify MethodCallExpr nodes, recording all method invocation names called within a method. This serves as the foundation for building the dependency graph.

Gemini Embedding Model (GeminiEmbeddingService)

Acts as a custom wrapper implementing Spring AI's EmbeddingModel. It coordinates with the Google Gemini API (model: models/gemini-embedding-001) via RestClient, providing a 768-dimension vector. It optimizes vector search query matching by requesting RETRIEVAL_DOCUMENT task types during code ingestion and RETRIEVAL_QUERY task types during search.

Vector Search Wrapper (VectorSearchService)

Integrates with Spring AI's PgVectorStore. It inserts documents with metadata maps (entity ID, repository ID, path, entity type) and handles similarity search filtering, applying a 0.65 Cosine Similarity threshold to ensure high-recall, low-noise results.

3. Database Schema & Migration

The relational database schema is managed via Flyway migration scripts. Schema evolution is decoupled from Hibernate's automatic table generation (which is set to validate only) to ensure production safety. Below is the relational structure of the database:

Table Name	Key Columns & Types	Purpose / Role
users	id (BIGINT PK) username (VARCHAR) email (VARCHAR)	Stores registered developer accounts.
repositories	id (BIGINT PK) owner_id (BIGINT FK) status (VARCHAR)	Tracks indexed repos and status (PENDING, INDEXING, DONE, FAILED).
code_entities	id (BIGINT PK) repo_id (BIGINT FK) vector_doc_id (VARCHAR)	Holds raw source snippets, line boundaries, and maps to the vector database UUID.
dependencies	from_entity_id (BIGINT PK, FK) to_entity_id (BIGINT PK, FK)	A composite key join table mapping caller-callee relationships.
vector_store	id (UUID PK) embedding (VECTOR) metadata (JSONB)	Spring AI table storing the 768-dimension vectors and indices.

4. Ingestion Concurrency & Error Isolation

To index medium-to-large repositories without locking up resource threads or crashing the server, CodeLens AI applies several key patterns in its Indexing Pipeline:

1. Multi-Threaded Executor Pool: Asynchronous methods run on a configured ThreadPoolTaskExecutor. Rather than taking up incoming Tomcat server threads, indexing tasks queue up on a pool with a core size of 4, max size of 8, and standard graceful shutdown configurations.

2. File-Level Transaction Isolation: A common bug in database ingestion is executing the whole repository download and database write inside a single transaction. In that scenario, if a single bad file fails, the entire import is rolled back. CodeLens AI isolates each file processing task using Spring AOP proxy self-injection (via @Autowired and @Lazy on self) and @Transactional. If one file fails (due to syntax issues or API rate limits), it commits successfully processed files and continues.

3. Asynchronous Batch Processing: Files are retrieved and compiled using CompletableFuture.runAsync(), meaning file contents are parsed and embedded concurrently, maximizing CPU core usage.

5. Critical Database & Local Optimizations

1. Index V2 Migration (Reverse Dependencies): The initial database schema set up a composite primary key on dependencies(from_entity_id, to_entity_id). Since PostgreSQL uses the leftmost columns for composite index searches, queries checking 'who depends on this method?' were doing full table scans. Flyway V2 added idx_deps_to on dependencies(to_entity_id) to optimize BFS impact analysis lookup times.

2. Auto-loading .env File: Running Spring Boot locally via mvn spring-boot:run doesn't natively import variables from .env files. We implemented an automatic dotenv loader inside the main class that injects env values directly into System properties before Spring starts. This keeps keys safe and local.

3. Test Profile Isolation: For unit tests, we configure H2 in-memory databases. H2 does not support PostgreSQL's pgvector extension. By using Spring active profiles, we marked VectorStoreConfig with @Profile('!test'), skipping schema generation on H2 and ensuring test suites run quickly and cleanly.